MiniDash Reflection

Selina Serat
CSCI 665
Project: MiniDash Track C: Live Order Tracking
October 29, 2026

I decided to create my MiniDash using Python, FastAPI, and SQLite. I created the front end using HTML, CSS, and JavaScript. The user will be able to see restaurants and view their menus. They can then place an order and track their delivery. I used ChatGPT to learn more about how to code certain aspects of my project and how to fix errors. When starting off with my project, I used ChatGPT to create my database and endpoints. I felt as if breaking my project down into smaller chunks allowed me to have a better understanding of what I was coding.

Something I learned while coding with ChatGPT is that sometimes the code will not work as intended. When I first tried my website, it wasn't displaying the food properly. I noticed that some foods did not have the right emojis and showed up with a price of "$NaN". I had to go back to ChatGPT and explain what was going wrong and edit my JavaScript code to display the correct information from the menu and price.

When I began testing my live order tracking, I ran into another problem. Once I placed an order, my website would say "Method Not Allowed". I realized that AI was providing me with the wrong endpoint to use for my order tracking. So I went back and looked at FastAPI's documentation for my tracking endpoint. Once I changed the endpoint and ran my website, it worked. Another problem I had with the AI-generated code was when I first tried to place an order, I was getting a 422 error because the checkout code wasn't sending a delivery address, which my backend required. After looking at the error and asking ChatGPT what was wrong, I updated the checkout code to ask for an address. Once I entered an address, I was able to place an order successfully.

While learning more about APIs, I discovered that you can only send certain information through your request. When I was testing order statuses, I put in an incorrect value and got a 422 error. After using AI to help figure out what was causing my error, I found out that my back end only accepted specific statuses. I was able to take my order through each stage, from PLACED to DELIVERED. This helped me understand more about the front end and back end of my website. I also learned how to test my code. ChatGPT helped me write automated tests, which I ran and checked throughout development. I finished my project with 21 tests passing. I manually tested my website as well by navigating through the site and making sure that my order tracked correctly. I also used Git and GitHub to commit my code and track my changes.

I believe that using AI allowed me to better understand my code because I could ask any question and it would give me ideas on what to do next. I told the AI to include lots of comments on any code it creates, explaining what the code does. That way, whenever I used AI-generated code, I would read the comments along the way and

understand what every part of the code does. However, there were times when I wasn't being clear with what I wanted from the prompt, so some of ChatGPT's code wasn't accurate for my application. As I continued working on the project, I learned to be more specific with my prompts. At first, I would ask ChatGPT to help me create entire parts of the application. Later on, I started explaining exactly what wasn't working and showing the errors I was getting. I found that this helped me get more useful answers. I think AI was most helpful when I needed to understand an error or figure out what to do next. It was less helpful when it gave me code without fully understanding how my application was already set up. One decision I made myself was to add a front end so users could actually interact with MiniDash instead of only using the API documentation. I wanted to be able to see the order tracking working on an actual website. Even though ChatGPT helped me write the code, I decided what I wanted the website to do and how I wanted users to interact with it.

Using AI helped me gain more knowledge about how to code using FastAPI and SQLite, as well as how to work on the front end, which is something I don't have much experience with. I also learned that you can't just rely on AI to code for you. You still have to know what you want to happen with your code and debug it when it isn't working, as well as understand what the code is doing rather than blindly trusting that AI is doing what it's supposed to.

Although my 21 automated tests passed and I manually tested the website, I know there are still things I haven't fully tested. For example, I haven't tested how the website would work with multiple customers placing orders at the same time. If I had more time, I would test that and make sure the tracking updates correctly for each order without mixing up information between customers. I feel like this project helped me become more confident with debugging and understanding how different parts of an application work together.